Simulación paralela de incendios forestales del Gran Chaco con OpenMP, MPI y un modelo híbrido

Autora: Lizzy Leyla Baron Barrero | **Materia:** TEL-420 Sistemas Paralelos, UAJMS Yacuiba | **Docente:** Ing. Elias Cassal Baldiviezo

I. RESUMEN Y ABSTRACT

Resumen. Los incendios forestales amenazan el Gran Chaco boliviano y el departamento de Tarija, y estudiar su propagación exige simular muchos escenarios. Este trabajo presenta un autómata celular estocástico con versiones secuencial, OpenMP, MPI e híbrida, y El Vigía, un predictor Monte Carlo que genera mapas de riesgo. Se ejecutan en un clúster Docker de tres nodos y se integran en el videojuego educativo Guardianes del Chaco. Como el número aleatorio de cada celda depende solo de la semilla, el paso y la celda, todas las versiones producen el mismo checksum. En una Dell OptiPlex 7010 (Core i7-13700), OpenMP alcanza un speedup de 12,07 con 16 hilos para n=2048 (eficiencia 0,75) y de 9,19 para n=512; la escalabilidad débil conserva una eficiencia de 0,79 con 16 hilos. MPI llega a 5,64 con seis procesos y el Vigía híbrido a 9,06 con tres procesos y cuatro hilos. Los tres nodos comparten la misma CPU física, por lo que los resultados de MPI no reflejan una red real. Los resultados son experimentales y educativos.

Abstract. Wildfires threaten the Bolivian Gran Chaco and the department of Tarija, and studying their spread requires simulating many scenarios. This work presents a stochastic cellular automaton with sequential, OpenMP, MPI and hybrid versions, and El Vigía, a Monte Carlo predictor that produces risk maps. They run on a three-node Docker cluster and are integrated into the educational video game Guardianes del Chaco. Because each cell's random number depends only on the seed, the step and the cell, all versions produce the same checksum. On a Dell OptiPlex 7010 (Core i7-13700), OpenMP reaches a speedup of 12.07 with 16 threads for n=2048 (efficiency 0.75) and 9.19 for n=512; weak scaling keeps an efficiency of 0.79 with 16 threads. MPI reaches 5.64 with six processes and the hybrid Vigía 9.06 with three processes and four threads. The three nodes share the same physical CPU, so the MPI results do not reflect a real network. The results are experimental and educational.

Palabras clave: autómata celular, incendios forestales, OpenMP, MPI, speedup, Amdahl, Gustafson.

Keywords: cellular automata, wildfire, OpenMP, MPI, speedup, Amdahl, Gustafson.

II. INTRODUCCIÓN Y CONTEXTUALIZACIÓN REGIONAL

Yacuiba se ubica en el Gran Chaco boliviano, en el departamento de Tarija, junto a la Serranía del Aguaragüe. Es una región de bosque seco y pastizales con una estación seca marcada, donde un incendio forestal puede extenderse con rapidez y afectar a la fauna silvestre, la vegetación y las comunidades cercanas. [PENDIENTE: dato o fuente oficial sobre incendios en Tarija o el Chaco (superficie afectada o focos de calor), por ejemplo de la ABT, el SERNAP o la Gobernación de Tarija.]

Simular la propagación del fuego con un autómata celular exige actualizar n × n celdas en cada paso, y estudiar el riesgo exige repetir la simulación muchas veces con distintas semillas (método Monte Carlo). Ambos costos crecen rápido con el tamaño del mapa y con el número de escenarios, y ambos admiten paralelismo: las celdas de un mismo paso son independientes entre sí gracias al doble buffer, y las simulaciones Monte Carlo son independientes entre sí.

Este trabajo presenta (1) un motor de simulación secuencial que sirve de referencia, (2) versiones paralelas con OpenMP, MPI y un modelo híbrido, (3) El Vigía, un predictor Monte Carlo que produce mapas de riesgo y permite comparar escenarios con y sin cortafuegos, (4) una infraestructura reproducible de tres nodos con Docker y (5) la integración del motor en Guardianes del Chaco, un videojuego educativo cooperativo orientado a la prevención de incendios y la protección de la fauna. El objetivo técnico es medir la aceleración, la eficiencia y la escalabilidad de cada modalidad y contrastarlas con las leyes de Amdahl y Gustafson. Los resultados son experimentales y educativos; no son predicciones oficiales de incendios reales.

El documento se organiza así: la Sección III presenta el marco teórico, la IV la arquitectura e infraestructura, la V la implementación paralela, la VI los resultados experimentales y la VII las conclusiones.

III. MARCO TEÓRICO Y ESTADO DEL ARTE

A. Taxonomía de Flynn: Flynn clasifica los sistemas según sus flujos de instrucciones y de datos en SISD, SIMD, MISD y MIMD [6]. La versión secuencial es SISD; las versiones paralelas son MIMD con el modelo SPMD, con memoria compartida (OpenMP) y distribuida (MPI): procesadores independientes ejecutan el mismo programa sobre porciones distintas de los datos.

B. Modelo de memoria: en memoria compartida (OpenMP), los hilos de un mismo proceso ven un único espacio de direcciones y deben evitar condiciones de carrera; aquí se evitan con el doble buffer, que separa la lectura del paso t de la escritura del paso t+1. En memoria distribuida (MPI), cada proceso tiene su propia memoria y los datos se mueven con mensajes explícitos [7]. El modelo híbrido usa MPI entre procesos y OpenMP dentro de cada uno; el Vigía inicializa MPI con MPI_THREAD_FUNNELED, de modo que solo el hilo principal hace llamadas MPI.

C. Redes de interconexión: en un clúster, el costo de enviar un mensaje de m bytes se aproxima por una latencia fija más un término proporcional a m dividido por el ancho de banda. En este proyecto la red es una red bridge virtual de Docker dentro de una sola máquina física, por lo que su latencia y su ancho de banda no representan los de una red Ethernet real. [DELL: medir la latencia entre contenedores si se quiere cuantificar.]

D. Leyes de Amdahl y Gustafson: con una fracción secuencial f, la aceleración máxima con p procesadores es S(p) = 1 / (f + (1 − f)/p) [1]; si el problema crece con p, la aceleración escalada es S(p) = p − f(p − 1) [2]. La eficiencia es E(p) = S(p)/p.

E. Estado del arte: los autómatas celulares se usan para modelar la propagación de incendios con efecto del viento y del combustible [8]. Las simulaciones Monte Carlo independientes son un caso de paralelismo casi trivial. [PENDIENTE: 2 o 3 trabajos sobre simulación paralela de incendios (MPI, OpenMP o GPU) con su referencia.]

IV. DISEÑO DE LA ARQUITECTURA E INFRAESTRUCTURA

A. Clúster: 3 contenedores Docker (master, worker1, worker2), red bridge, SSH sin contraseña, Ubuntu 22.04, 8 GB de RAM y un límite de CPUs por nodo configurable (4 por defecto; 16 en las mediciones de la Dell).

B. Hardware de medición: Dell OptiPlex Tower Plus 7010 con Intel Core i7-13700 de 13.ª generación (16 núcleos, 24 hilos lógicos, hasta 5,2 GHz, 30 MiB de caché L3) y 31 GiB de RAM; Docker 29.7.0 en el host y, dentro de los contenedores, Ubuntu 22.04 con gcc 11.4.0 y Open MPI 4.1.2. [verificar en la ficha de Intel: 8 núcleos de rendimiento y 8 de eficiencia.]

C. Asignación de recursos: cada contenedor recibe hasta CPUS_NODO CPUs (4 por defecto; 16 en las mediciones de la Dell) y 8 GB de RAM (parámetros cpus y mem_limit de infrastructure/docker-compose.yml). Los tres nodos comparten la CPU física del host, de modo que el clúster reproduce la lógica de un sistema distribuido pero no el costo de una red real. El archivo hostfile lista los tres nodos y los procesos MPI se reparten con --map-by node (un proceso por nodo).

D. Diagrama del clúster: la Fig. 1 muestra el host, la red bridge interna y los tres contenedores (master, worker1, worker2) con sus CPUs y su RAM.

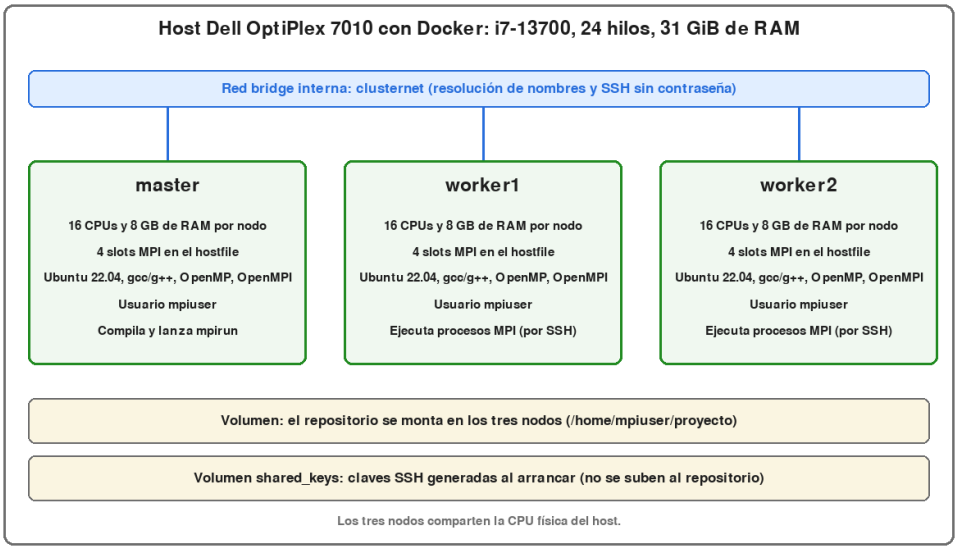


Fig. 1. Arquitectura del clúster de 3 nodos sobre el host.

V. IMPLEMENTACIÓN ALGORÍTMICA PARALELA

A. Modelo y algoritmo base.

El incendio se modela como un autómata celular estocástico sobre una malla de n x n celdas con vecindad de Moore (8 vecinas) y doble buffer: el paso t+1 se calcula leyendo únicamente el paso t, lo que elimina dependencias dentro de un mismo paso.

Cada celda puede estar vacía, con vegetación, quemando, quemada, con cortafuegos o con agua, y solo la vegetación puede encenderse. Una celda en llamas pasa a quemada con probabilidad p_apagado = 0,25 por paso. Para una celda de vegetación, cada vecina k en llamas aporta una probabilidad de contagio p_k = min(0,95; p_base · f_c · f_h · f_v,k), con p_base = 0,30. El factor de combustible es f_c = combustible/255, el de humedad f_h = 1 − 0,8 · humedad/255, y el de viento f_v,k = d_k · exp(1,5 · w · cos θ_k), donde w es la fuerza del viento, θ_k el ángulo entre la dirección del viento y la dirección vecina → celda, y d_k vale 1 para vecinas ortogonales y 0,7071 para diagonales.

La probabilidad total de ignición combina a todas las vecinas en llamas, P = 1 − Π(1 − p_k), y la celda se enciende si u < P. El número aleatorio u se obtiene de SplitMix64 aplicado a la terna (semilla, paso, índice de celda), de modo que depende solo de esos tres valores y nunca del hilo o proceso que calcula la celda. Esta decisión de diseño hace que todas las versiones produzcan exactamente el mismo resultado y permite verificarlas con un checksum del estado final.

Las acciones del jugador son APAGAR (quemando → quemada) y CORTAFUEGO (vegetación → cortafuegos), y se aplican antes de calcular cada paso.

B. OpenMP (memoria compartida): las filas de la malla se reparten entre los hilos con la cláusula schedule(runtime), lo que permite comparar las políticas static, dynamic y guided sin recompilar. Gracias al doble buffer, los hilos escriben en el buffer de destino sin sincronización dentro del paso.

C. MPI (memoria distribuida): la malla se descompone por bloques de filas, uno por proceso. Las primeras (n mod p) filas-bloque llevan una fila extra, de modo que la diferencia de carga entre procesos es de una fila. En cada paso, cada proceso aplica primero las acciones del jugador sobre sus filas y luego intercambia la fila frontera con sus vecinos superior e inferior mediante MPI_Isend y MPI_Irecv no bloqueantes seguidos de MPI_Waitall, lo que evita interbloqueos. Después calcula solo sus filas. Al final, las métricas globales se obtienen con MPI_Reduce (suma de tres contadores globales) y el estado final se reúne en el proceso 0 con MPI_Gatherv. Limitación actual: cada proceso reserva la malla completa y solo calcula su bloque; la comunicación es la mínima necesaria, pero la memoria por proceso no se reduce (mejora prevista: bloque local más dos filas halo).

D. Híbrido y Vigía: el Vigía ejecuta simulaciones Monte Carlo independientes con distintas semillas. Las simulaciones se reparten entre procesos MPI y, dentro de cada proceso, los hilos OpenMP procesan su parte. Los resultados se acumulan en un mapa de probabilidad de riesgo de quema por celda. Al ser simulaciones independientes, casi no requieren comunicación: la única es una reducción final. La simulación s se asigna al proceso (s mod p), los hilos de cada proceso toman simulaciones con schedule(dynamic,1) y cada hilo usa un mapa acumulador y un doble buffer privados, por lo que no hay sincronización dentro de una simulación. Los acumuladores de los hilos se fusionan en una sección crítica y los de los procesos con MPI_Reduce. La semilla de cada simulación se deriva con SplitMix64 solo de la semilla base y del índice s, no del proceso ni del hilo, por lo que el mapa de riesgo no debería depender de cómo se reparta el trabajo (verificado en la Dell con n=512, 48 simulaciones y 100 pasos: el mismo checksum eddb391a611603aa con 1 y 3 procesos y con 1 y 4 hilos). El Vigía acepta además un archivo de acciones (--acciones), con el mismo formato que el motor secuencial, para comparar el riesgo con y sin cortafuegos; sin ese archivo el resultado es idéntico al anterior (checksum de referencia verificado con n=128, 8 simulaciones y 50 pasos: 3fcc85cb0166e9e5). La Fig. 2 compara el mapa de riesgo sin y con una barrera completa.

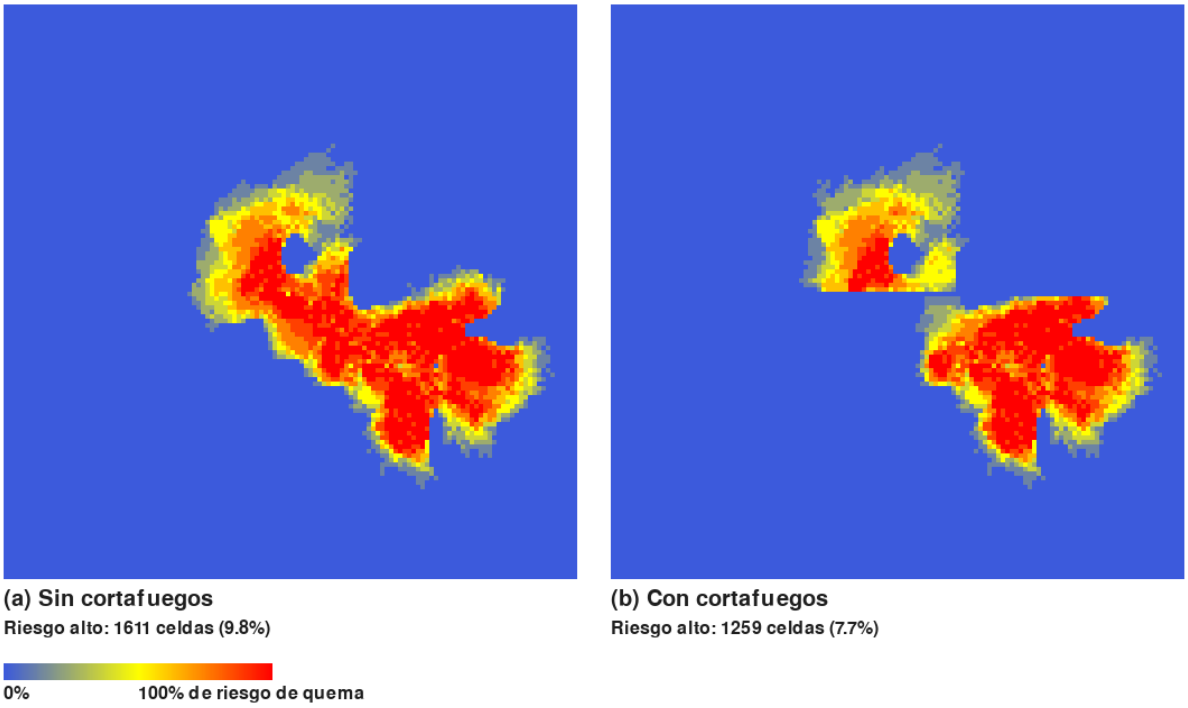


Fig. 2. Riesgo de quema del Vigía (a) sin y (b) con cortafuegos; mapa de 128 celdas, 8 simulaciones, 50 pasos.

E. Balanceo de carga: en OpenMP, schedule(runtime) permite comparar static, dynamic y guided sin recompilar [DELL: resultado]. En MPI, el reparto por bloques de filas deja una diferencia máxima de una fila entre procesos, aunque el costo real por fila puede variar con la cantidad de fuego. En el Vigía, el reparto cíclico de simulaciones entre procesos y la planificación dinámica entre hilos compensan simulaciones de duración desigual.

F. Corrección: checksum idéntico en todas las versiones (n=512, semilla 42, 100 pasos: c5dad814f7cbeaff).

G. Integración con el videojuego: el motor secuencial (`fire_seq`) se ejecuta dentro del contenedor `master` y el videojuego en Python (pygame) lo invoca una vez por ronda de cinco pasos. Las acciones de los jugadores (cortafuegos y agua) se escriben en un archivo de acciones que el motor lee en el paso indicado, y el estado del mapa se guarda y se recupera entre rondas, de modo que la simulación continúa sin recalcularse desde cero. El cálculo corre en un hilo aparte para que la interfaz no se bloquee. Cada nivel de la campaña (Bosques de Yacuiba, Serranía del Aguaragüe, Región de Tarija y El gran incendio) define su propio tamaño de mapa (96, 112, 128 y 128 celdas de lado), su semilla y la dirección e intensidad del viento. El Vigía recibe los mismos parámetros de viento que el fuego real, para que su mapa de riesgo Monte Carlo sea coherente con la partida. Los recursos del bombero (tanque de seis cargas con recarga junto al río y tiempo de recuperación del cortafuegos) y el compañero

controlado por reglas se implementan en el cliente y no modifican el motor, por lo que el checksum de referencia no cambia. La Fig. 3 muestra la interfaz con las habilidades del bombero y la veterinaria.



Fig. 3. Interfaz del juego con las habilidades de los personajes.

VI. RESULTADOS EXPERIMENTALES Y ANÁLISIS DE RENDIMIENTO

A. Protocolo: tamaños n=512, 1024 y 2048 con 100 pasos, hilos OpenMP 1, 2, 4, 8 y 16, y media de 5 repeticiones por configuración, todo en la Dell OptiPlex 7010; procesos MPI 1, 2, 3 y 6 (n=2048, una medición). Métricas: T_p, S_p = T_1/T_p, E_p = S_p/p.

B. Reproducibilidad: `scripts/dell_medicion.sh`, semilla fija, repositorio con un comando.

C. Huella de memoria: RSS máximo de la versión secuencial (medido con /usr/bin/time): 4,07 MiB (n=512), 7,96 MiB (n=1024) y 22,61 MiB (n=2048).

D. Escalabilidad fuerte: T_seq = 0,1452 s (n=512), 0,5588 s (n=1024) y 2,2543 s (n=2048). Speedup S y eficiencia E de OpenMP (media de 5 repeticiones, 100 pasos, checksum idéntico):

Hilos	n=512 S / E	n=1024 S / E	n=2048 S / E
1	1,027 / 1,027	1,023 / 1,023	1,038 / 1,038
2	1,926 / 0,963	1,955 / 0,977	2,027 / 1,014
4	3,636 / 0,909	3,816 / 0,954	4,045 / 1,011
8	6,748 / 0,843	7,242 / 0,905	7,870 / 0,984
16	9,187 / 0,574	11,871 / 0,742	12,070 / 0,754

El speedup crece con el tamaño del problema: con 16 hilos pasa de 9,19 (n=512) a 12,07 (n=2048). Las eficiencias ligeramente mayores que 1 (hasta 1,038) se atribuyen a variación de medición y a efectos de caché, no a un comportamiento superlineal. La caída de eficiencia con 16 hilos se discute en VI.G. Ajuste de Amdahl: la fracción serial efectiva es f=0,046 (n=512), 0,022 (n=1024) y 0,020 (n=2048), con speedup máximo teórico de 21,7, 45,5 y 50,0. Con 16 hilos Amdahl predice 9,47, 12,03 y 12,31 frente a 9,19, 11,87 y 12,07 medidos. f incluye el efecto del ancho de banda de memoria, no solo el código secuencial. La Fig. 4 resume el speedup y la eficiencia.

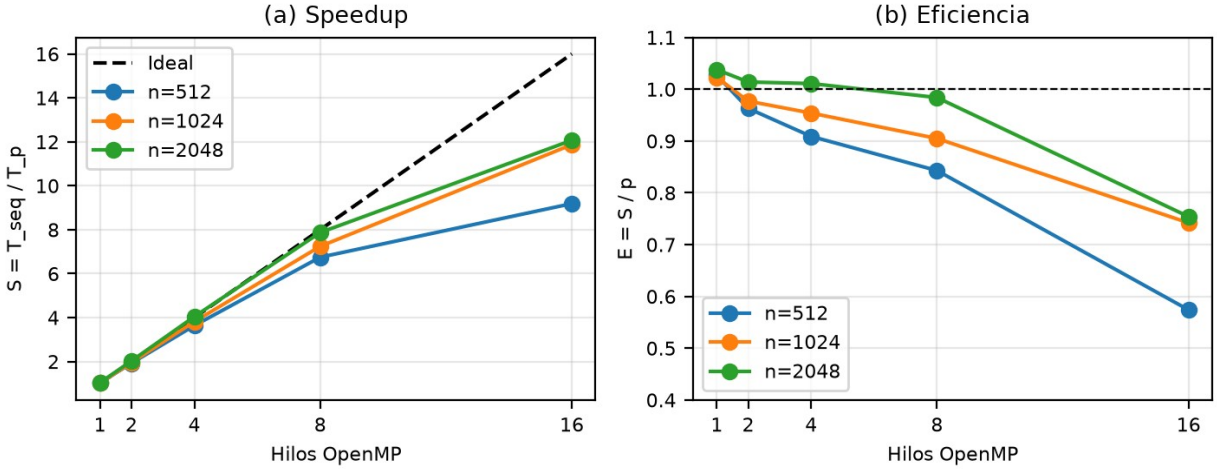


Fig. 4. Escalabilidad fuerte de OpenMP: (a) speedup y (b) eficiencia.

E. Escalabilidad débil: con la carga por hilo constante (n=512, 724, 1024, 1448 y 2048 para 1, 2, 4, 8 y 16 hilos) el tiempo se mantiene entre 0,147 s y 0,151 s hasta 8 hilos y sube a 0,185 s con 16. La eficiencia débil es 0,993, 0,997, 0,972 y 0,792 para 2, 4, 8 y 16 hilos, y el speedup escalado con 16 hilos es 12,67. La fracción serial escalada media es α=0,068, con la que Gustafson predice unos 14,98 con 16 hilos; lo medido queda por debajo porque α no es solo código secuencial: incluye ancho de banda de memoria y efectos de hyperthreading, y con 16 hilos sube a 0,222 (Fig. 5).

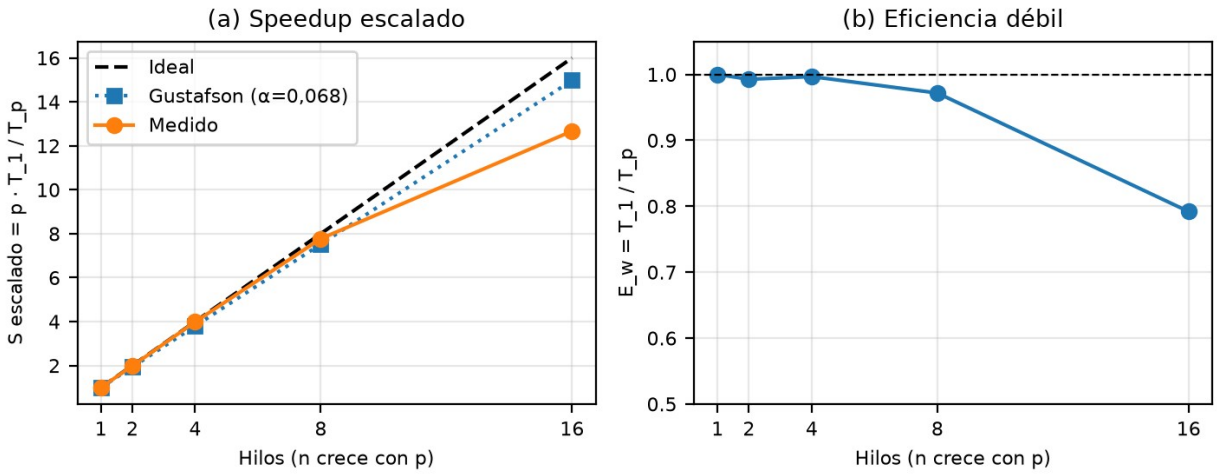


Fig. 5. Escalabilidad débil: (a) speedup escalado frente a Gustafson y (b) eficiencia débil.

F. MPI e híbrido: MPI sobre los 3 contenedores (n=2048, 100 pasos, una medición por configuración, checksum 82545e9347fe1d2b en todas):

Procesos	T (s)	S frente a T_seq	E
1	2,166	1,04	1,04
2	2,138	1,05	0,53
3	0,765	2,95	0,98
6	0,399	5,64	0,94

Con 2 procesos casi no hay mejora y con 3 el tiempo cae a menos de la mitad. [PENDIENTE: explicar esta irregularidad; hay que repetir la medición y revisar cómo se reparten los procesos entre nodos.] Vigía híbrido (n=512, 48 simulaciones, 100 pasos, mismo checksum eddb391a611603aa en todas):

Procesos	Hilos	T (s)	S frente a 1x1
1	1	6,541	1,00
1	4	6,603	0,99
3	1	2,298	2,85
3	4	0,722	9,06

Con 3 procesos y 4 hilos (12 unidades de trabajo) la eficiencia es 0,75. Que 4 hilos no mejoren nada con 1 proceso es inesperado. [PENDIENTE: verificar si OpenMP usó realmente los 4 hilos en ese caso.]

G. Cuellos de botella: ancho de banda de memoria, hyperthreading, los 3 nodos comparten la misma CPU física, red virtual sin latencia real.

H. Datos preliminares (laptop, no oficiales): Vigía con cortafuegos (laptop; mapa de 128 celdas, 8 simulaciones, 50 pasos): el porcentaje de celdas con riesgo mayor al 50 % baja de 9,8 % (1611 celdas) a 7,7 % (1259 celdas) al agregar una barrera completa; es una demostración cualitativa que se repetirá en la Dell.

VII. CONCLUSIONES E IMPACTO SOCIAL Y PRODUCTIVO

El número aleatorio derivado de (semilla, paso, celda) hizo que las versiones secuencial, OpenMP, MPI y el Vigía dieran resultados idénticos, verificados con checksum. En la Dell, OpenMP escala bien hasta 8 hilos (eficiencia entre 0,84 y 0,98 según n) y cae con 16 hilos (entre 0,57 y 0,75), con mejor comportamiento cuanto mayor es el problema; el ajuste de Amdahl da fracciones seriales efectivas entre 0,020 y 0,046. La escalabilidad débil conserva una eficiencia de 0,79 con 16 hilos, por debajo de la predicción de Gustafson, y los cuellos de botella señalados son el ancho de banda de memoria y el hyperthreading (16 núcleos y 24 hilos lógicos). MPI alcanzó 5,64 con seis procesos, pero con una sola medición, una irregularidad con 2 procesos todavía sin explicar y nodos que comparten CPU, por lo que no mide el costo de una red real. El Vigía híbrido alcanzó 9,06 con tres procesos y cuatro hilos. Trabajo futuro: CUDA, más regiones, juego completo, otras redes.

Impacto social y productivo: el proyecto ofrece una herramienta abierta para sensibilizar sobre la prevención de incendios y la protección de la fauna del Chaco, y permite comparar escenarios, por ejemplo con y sin cortafuegos. [PENDIENTE: valorar con datos o actores locales; los resultados son experimentales y no constituyen predicciones oficiales.]

VIII. REFERENCIAS

[1] Amdahl, G. M., "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

[2] Gustafson, J. L., "Reevaluating Amdahl's law," Commun. ACM, 1988.

[3] OpenMP Architecture Review Board, OpenMP Application Programming Interface.

[4] MPI Forum, MPI: A Message-Passing Interface Standard.

[5] Steele, G. L. et al., "Fast splittable pseudorandom number generators," OOPSLA 2014 (SplitMix64).

[6] Flynn, M. J., "Very high-speed computing systems," Proc. IEEE, 1966. [verificar datos]

[7] Pacheco, P., An Introduction to Parallel Programming, Morgan Kaufmann, 2011. [verificar datos]

[8] Karafyllidis, I. y Thanailakis, A., "A model for predicting forest fire spreading using cellular automata," Ecological Modelling, 1997. [verificar datos]

[PENDIENTE: fuente oficial de datos regionales de incendios y 2 o 3 trabajos sobre simulación paralela de incendios.]